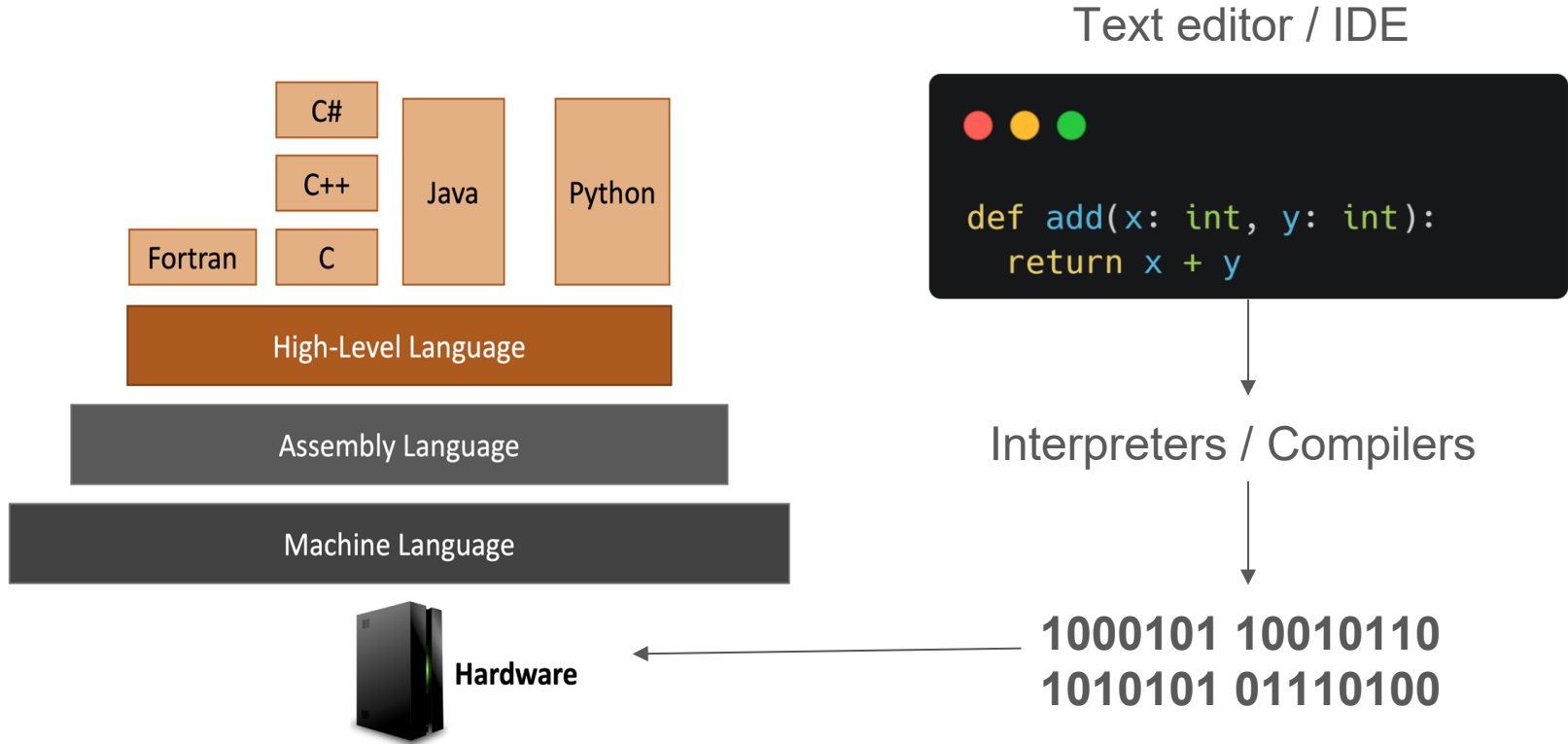


CHEM 273 Discussion 1

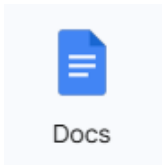
Usman Jamshed

High level programming overview



What is a text editor?

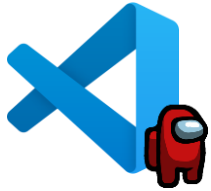
A program used to write and edit code.



Yes, these are technically text editors.

What is an IDE?

Buils on top of a text editor and has additional features built-in OR available via plugins: linting, debugger, build systems, version control.



eclipse



spyder

Might even have some fancy AI integrations to help write boiler plate or check logic.

Programming Environments

Essentially a digital workspace that can hold all your tools necessary for development.

Includes:

- Libraries / Packages
- Environment variables
- Compilers / Interpreters
- Debuggers



With respect to python there's 2 main ways to create an environment:



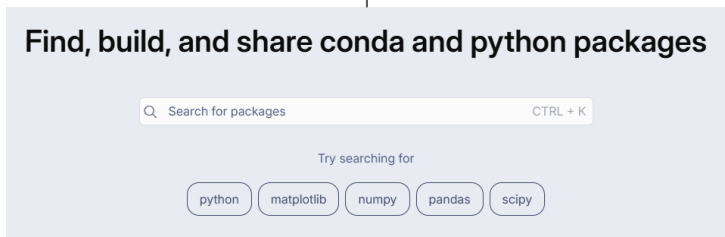
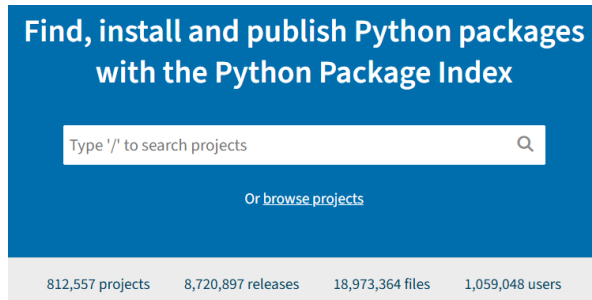
```
python -m venv .venv  
  
source .venv/bin/activate
```



```
conda create --name myenv python=3.11  
  
conda activate myenv
```

Installing packages

pip install numpy



conda install numpy



Developer Env

some_file.py

```
import numpy
import scipy
import matplotlib
```

Open up a blank jupyter notebook

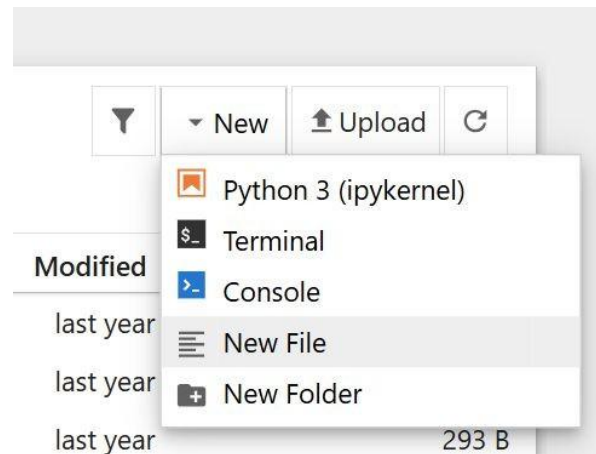
1. Open your terminal
2. Activate your Conda environment

```
conda activate msse-python
```

3. Start Jupyter Notebook

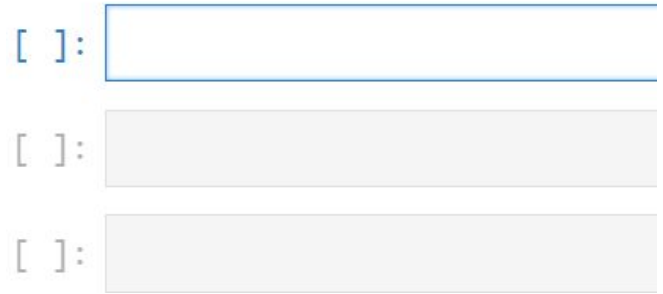
```
jupyter notebook
```

4. A browser window will open, navigate to the folder you want
5. Click New -> Python 3 to create a blank notebook



Jupyter Notebook

- Cells let you run specific parts of your code independently, without rerunning the entire notebook.
- To execute the code in a cell you can press the run button or use the keys: Shift + Enter
- To create a new cell you can use the plus in the toolbar or: Esc + B for a cell below and Esc + A for a cell above
- To run the current cell and create a new cell: Alt + Enter
- To save the notebook use Ctrl + S 



Try creating some new cells and saving your notebook

Import statements

Import statements are used to access external libraries or modules

- The syntax is: `import module_name`
- You can also rename a module:
- Or import only specific functions:
- Once you import a library you can call a specific function by using the syntax:
`module_name.method()`

```
import math
```

```
import pandas as pd
```

```
from math import sqrt
```

```
[2]:
```

```
math.log10(100)
```

```
[2]:
```

```
2.0
```

Import statements

If you get an error like this:

Try it out: Import the math library and take the square root of a number using `math.sqrt()`

```
-----  
ModuleNotFoundError                                Traceback (most recent call last)  
Cell In[4], line 1  
----> 1 import example_module  
  
ModuleNotFoundError: No module named 'example_module'
```

What's the fix?

- `!pip install example_module`
- You only need to install a package once, but you need to import at the beginning of each notebook or python file where you plan to use it

Variables

- Variables store info you want to use later
- Since Python is dynamically typed there is no need to declare variable types (this is different from other languages)
- Try to give descriptive names when possible
- Example:

<code>pi = 3.14</code>	float
<code>pi = [3, 1, 4]</code>	list
<code>pi = "yummy"</code>	string (str)

Try it out: Set variables with your name and favorite number and then print them using

```
print(name, favorite_num)
```

Comments

- Comments are lines of text are ignored when running code
- They can be used to explain code, or as placeholders
- To write a comment in Python preface the line with “#” or press Ctrl + /
- For multi-line comments you can use triple quotes

```
this is code  
# this is a comment
```

```
'''  
This is a multi-line comment.  
It can be used for longer explanations  
- point 1  
- point 2  
- point 3  
'''
```

Functions

- Functions are used to reduce repeating similar code blocks
- A function is defined using “def”
- Functions can take in parameters, or not
- The keyword “return” is used to return the output of a function

```
def my_function(parameter1, parameter2, parameter3):  
    # code  
    return answer
```

```
my_function("input1", "input2", "input3")
```

Functions

Here's a real example of a function

Defining the function:

```
def pluralize(word):  
    answer = word + "s"  
    return answer
```

Try it yourself: Write a function that sums two numbers.

When wouldn't this function work?

Using the function:

```
pluralize("cat")
```

```
[23]:
```

```
'cats'
```

```
pluralize("dog")
```

```
[24]:
```

```
'dogs'
```

Function Wrappers / Decorators

- A decorator is a function that modifies another function
- It uses the syntax of `@my_decorator` in front of the function you want to apply it to
- A common example is to use the timer wrapper to measure how long a function takes to complete

```
from my_timer import my_timer

@my_timer
def slow_math():
    return 1234567**12345

slow_math()
```

Total runtime: 0.014999999897554517 seconds

Where to look for information

- Use `dir(library)` to list all the available functions
- To find information about a specific function in a library you can use
`help(library.function)` or
`library.function?`
- When in doubt documentation for big libraries/modules is available online through a quick google search

```
import pandas as pd
```

```
dir(pd)
```

```
help(pd.unique)
```

```
pd.unique?
```

Reading excel files using pandas

1. Import pandas
2. Ensure the file you want to read is in the same folder as the notebook
3. Use the `pd.read_excel` function, passing in the filepath as a parameter
4. Examine the dataframe

Reading excel files using pandas

```
import pandas as pd

df = pd.read_excel("Data_set_0.xlsx")
df.head()
```

	Unnamed: 0	time	sample 0	sample 1	sample 2	sample 3	sample 4	sample 5
0	0	0.000	26.807584	17.692263	35.854515	34.766775	31.526637	21.458212
1	1	0.001	26.810342	15.909579	37.021849	35.485352	31.483660	22.094274
2	2	0.002	26.814777	15.261056	36.507977	36.068128	31.348556	23.026961
3	3	0.003	26.820926	16.093374	34.395095	36.512058	31.125543	23.970840
4	4	0.004	26.828819	18.265285	31.371029	36.816878	30.822213	24.639101

5 rows × 102 columns